Day 8 Task Report

Title: RAG System (The "Document Chatbot")

TASK-08: Architecting a Retrieval-Augmented Generation (RAG) Pipeline

---

Objective

The objective of this task is to build a Document Chatbot using a RAG (Retrieval-Augmented Generation) system. The system should read a document (PDF) and answer user questions based only on the content of that document.

The goal is to ensure the chatbot provides accurate and context-based answers, and avoids generating incorrect information.

Tools Used

- VS Code
- Git & GitHub
- MS Word

Libraries Used

- sentence-transformers : To generate embeddings
- faiss : To store and search embeddings efficiently
- PyPDF2 : To read PDF documents
- google-genai : To generate answers using Gemini
- dotenv : To manage API keys securely
- numpy :For numerical operations

Concepts Used

1. Retrieval-Augmented Generation (RAG)

A system that retrieves relevant data from a document and uses it to generate accurate answers.

---

2. Text Chunking

Breaking large text into smaller parts (chunks) for better processing and retrieval.

---

3. Embeddings

Converting text into numerical vectors using a machine learning model.

---

4. Vector Database (FAISS)

Stores embeddings and allows fast similarity search.

---

5. Cosine / Distance Similarity

Used to find the most relevant chunks based on the query.

---

6. Grounded Response

Ensuring the chatbot answers only from the provided context.

Code Explanation

The program works in the following steps:

1. Read the PDF file using PyPDF2
2. Extract text from all pages
3. Split text into smaller chunks
4. Convert chunks into embeddings using SentenceTransformer
5. Store embeddings in FAISS vector database

6. When user asks a question:
    - Convert query into embedding
    - Search FAISS for most relevant chunks
    - Send context + question to Gemini
7. Generate answer strictly based on context

Errors Faced

---

1. Mixing Two Embedding Methods

Problem:
Used both:

- SentenceTransformer
- Gemini embed API

Solution:
Removed Gemini embedding function and used only one method (SentenceTransformer).

---

2. File Reading Issue

Problem:
PDF text was not properly extracted.

Solution:
Used PyPDF2 correctly and ensured file path was valid.

---

3. Empty Query Issue

Problem:
Program crashed on empty input.

Solution:

Added input validation:

if not user_input.strip():

---

4. Irrelevant Answer Issue

Problem:

Sometimes answers were not accurate.

Weak context or improper chunking.

Solution:

Improved chunk size and ensured proper retrieval.

Output

```
(.venv) PS D:\Qualtech\Daily Tasks\Day8> py -3.10 rag-system.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|| 103/103 [00:00<00:00, 3389.67it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
-----------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Reading PDF...
Chunking text...
Creating embeddings (local model)...
Vector DB ready!

Ask your question (type 'exit' to quit): Give me information about Biometric-Frequency Identification

Answer: Security is handled via Biometric-Frequency Identification (BFI). The Aegis Layer uses the device's haptic feedback sensor to capture the "Resonance Frequency" of the use
r's cardiac rhythm (heartbeat). Each human heart vibrates at a unique frequency when under the 14.2 KHz "Pulse -Scan" emitted by the Vortex hardware. For initial setup and to map
 their "Resonance Profile," users must remain within 2 inches of the sensor for 60 seconds.
```

```
Ask your question (type 'exit' to quit): Explain Shard-Specific Responsibilities

Answer: Each of the 12 shards is hard-coded to manage specific system silos to prevent resource contention:
*   **Shards 01 - 03:** Hardware Abstraction Layer (HAL) and Interrupt Requests (IRQs).
*   **Shard 04:** I/O Buffer management and Peripheral synchronization.
*   **Shards 05 - 06:** Virtual Memory Paging and Swap-Space allocation.
*   **Shard 07:** The "L3 - Bridge," which manages communication between the CPU cache and the Pulse -Core.
*   **Shard 08:** Thread-level parallelism and load balancing.
*   **Shards 09 - 11:** The Aegis Security Layer and Wraith Firewall.
*   **Shard 12:** External Networking (TCP/IP/Vortex-Protocol).
```

```
Ask your question (type 'exit' to quit): What is Data Analysis ?

Answer: I DON'T KNOW.

Ask your question (type 'exit' to quit): exit
```